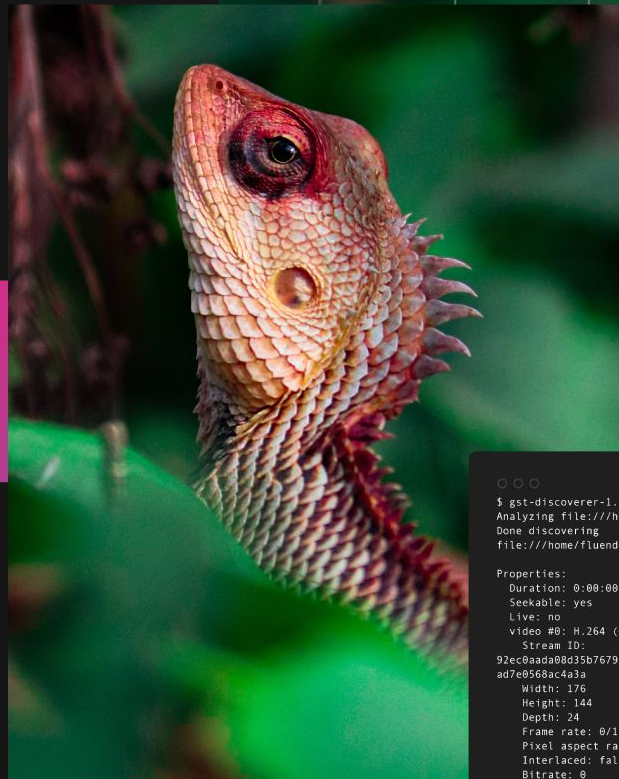


Gst.WASM

Season III



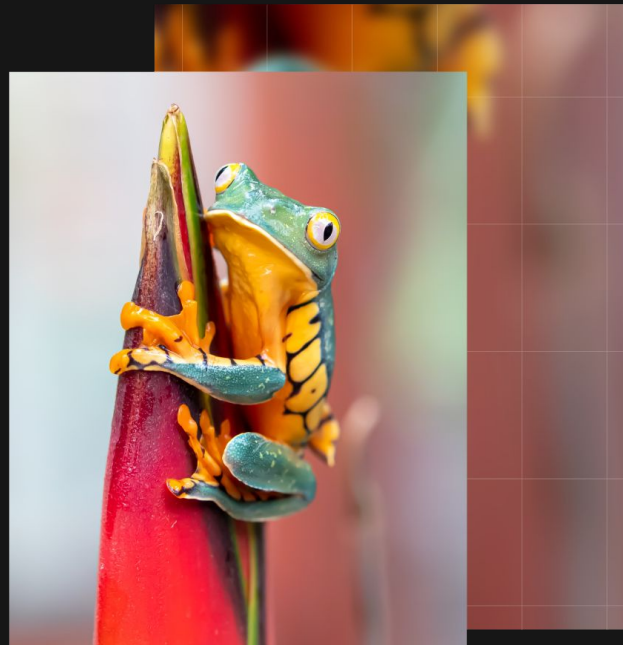
```
gst-discoverer-1.0 AUD_MW_E.264
Analyzing file:///home/fluendo/Videos/AUD_MW_E.264
Done discovering
file:///home/fluendo/Videos/AUD_MW_E.264

Properties:
  Duration: 0:00:00.000000000
  Seekable: yes
  Live: no
  video #0: H.264 (Constrained Baseline Profile)
  Stream ID:
92ec0aada08d35b7679f87a98465b4cf955fbad11d2c853Sec
ad7e0568ac4a3a
  Width: 176
  Height: 144
  Depth: 24
  Frame rate: 0/1
  Pixel aspect ratio: 1/1
  Interlaced: false
  Bitrate: 0
  Max bitrate: 0
```

Index

- Recap
- New Features
- Ongoing Features
- Real-life usage

Recap





| Overview

- Main goal was to improve the browser's **codec offer** by replacing the WebCodecs API with GStreamer
- Project started in **2023** with an initial port of GStreamer (GLib, libffi, etc) to Emscripten
- **Publicly** available at <https://github.com/fluendo/gst.wasm>
- Long term goal to **upstream everything**



| Features

- **GstWebRunner:** synchronization mechanism, single thread dispatcher
- **GstWebCanvas GstContext:** OpenGL-like, single context to defer execution in the same thread and same canvas
- **GstWebVideoFrame:** specific GstMemory for Web's VideoFrame



| Plugins (gst-plugins-web)

- **webcavassrc** To gather <canvas> image content as a source element
- **webcavassink** To render image/video content into <canvas>
- **webcodecsviddec[h264][sw,hw]** To decode using Web's WebCodecs API

New features



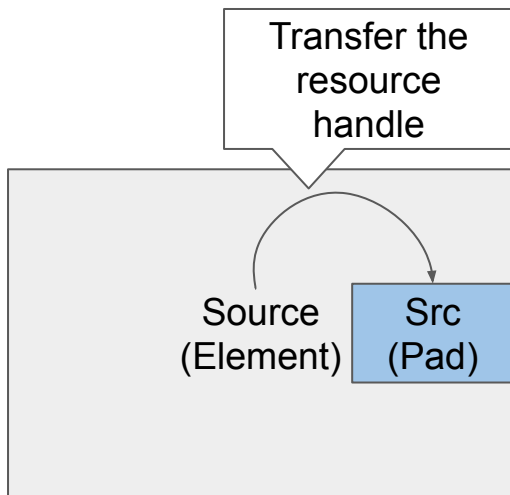


| Synchronization by transfer

- pthreads are implemented via a Web's WebWorker API
- WebWorkers communication mechanism is done by message passing, i.e. `postMessage` and `onMessage`
- WebWorkers operate on transferred objects by posting a message
- Not every object can be manipulated in a WebWorker, only transferable objects
- Transferable objects are just a few (`...`, `ReadableStream`, `WebTransportReceiveStream`, `WebTransportSendStream`, etc).



| Synchronization by transfer in GStreamer



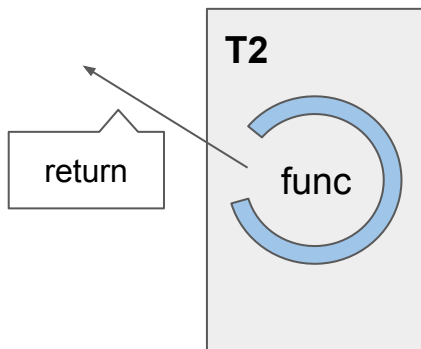
- **Application thread (Element):**
Resources setup, configuration and cleanup (`_init`, `_deinit`, `_set_property`, `_get_property`, `_setup`, `_change_state`, etc).
- **Streaming thread (Pad):**
Resource handle operations



Threads and WebWorkers

libc

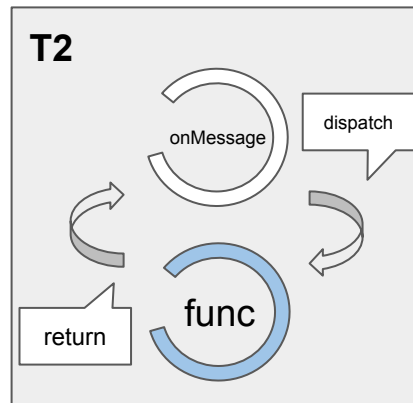
T1
pthread_create(func)



- GTask on pause waits until the state changes (GCond wait, lock, etc)
- On return the thread is **destroyed**

Emscripten

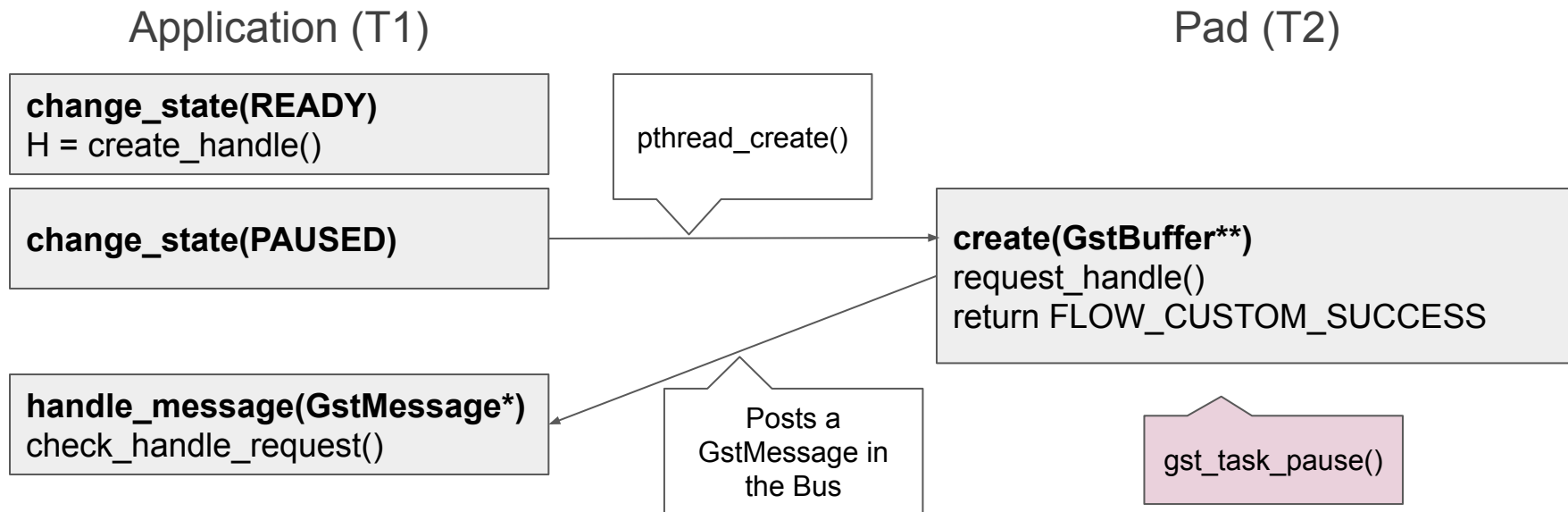
T1
pthread_create(func)



- On return the control of the thread **goes back** to the WebWorker's message handling

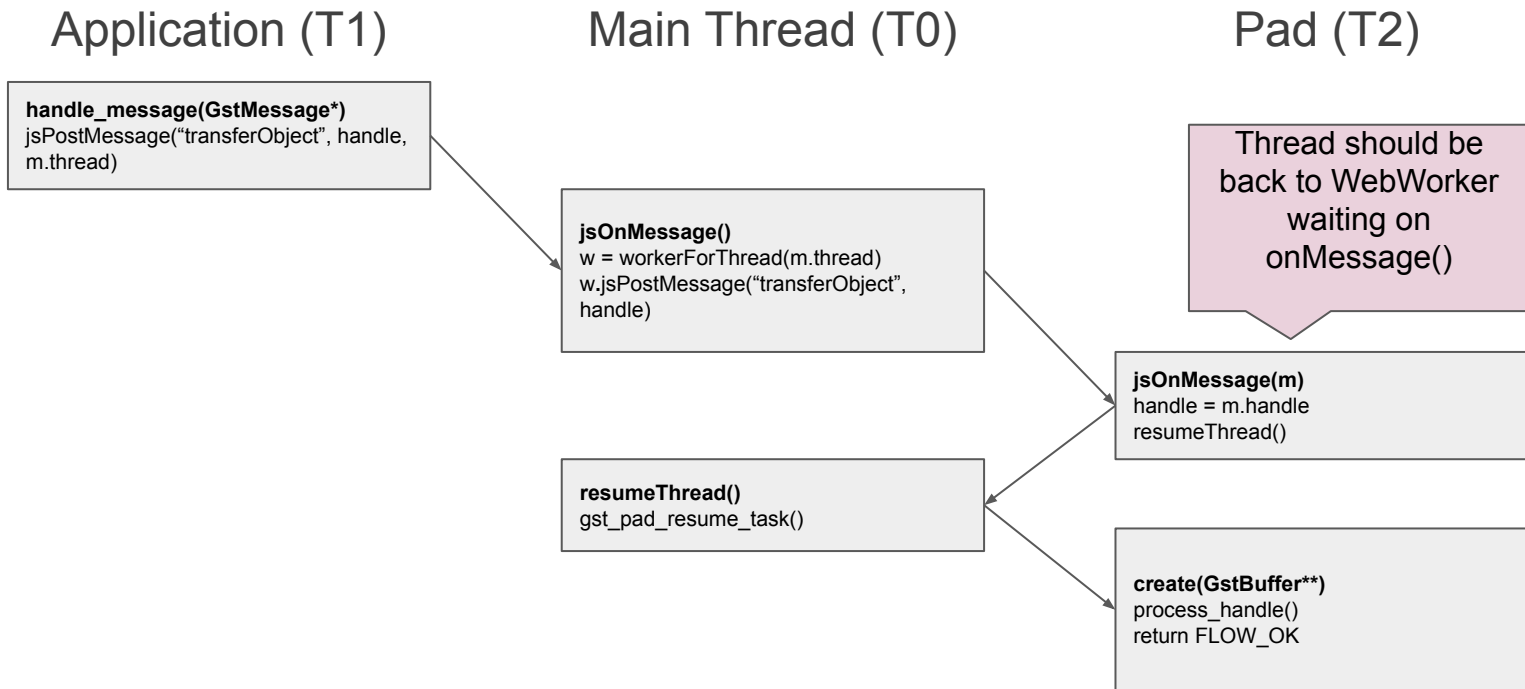


Requesting a Transfer





Transferring a handle





| GstWebTask, GstWebTaskPool

- GstTask implementation does not release the thread (GCond/GMutex)
- Subclass GstTask and GstTaskPool to **properly pause and resume** a Task by “releasing” it
- Must be used in the bus listener (parent element / application) on `GST_MESSAGE_STREAM_STATUS`



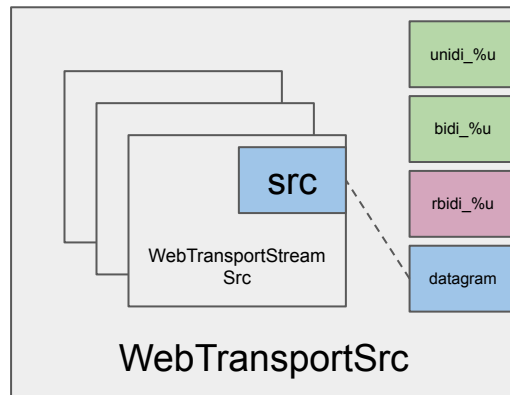
| GstWebTransferableInterface

- Logic to receive objects
- Logic to send object
- Logic to validate object by an id (string)
- Works for requesting from a child element
- Works for requesting from another element in the pipeline



| WebTransportSrc

- WebTransport API
- Update to WebSockets
- Transmitting data between client and server using HTTP/3 Transport (QUIC).
- Supports multiple streams
- Out-of-order delivery (datagrams) (**always pad**)
- Unidirectional streams (**sometimes pad**)
- Bidirectional streams (**request pad**)





| Other elements

- **GstWebDownload**
 - Copy data to a buffer with `VideoFrame.copyTo` and push it
- **GstWebUpload**
 - Wrap buffer data to a `VideoFrame` and push it
- **GstWebCodecsAudioDec**
 - AAC Support
- **GstWebStreamSrc**
 - Streams API Fetch

Ongoing Features





I ORC

- WASM has a SIMD extension
- 200+ instructions
- Subset of NEON / SSE
- Ongoing MR at

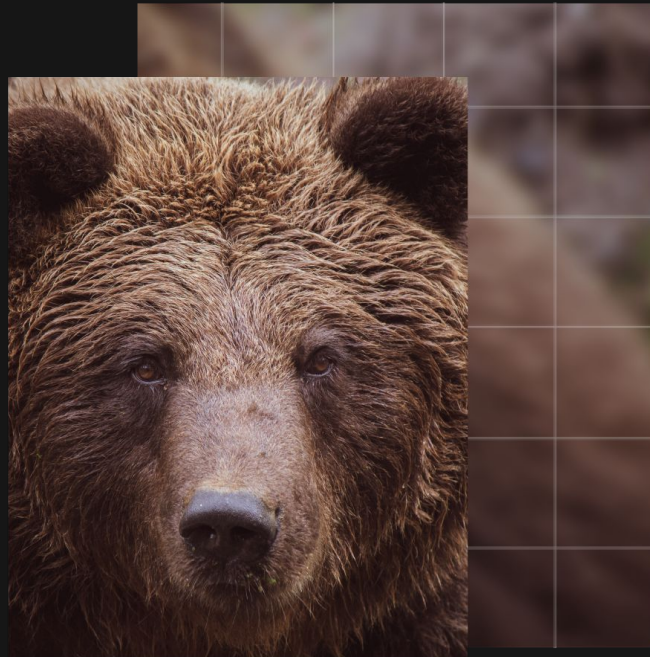
https://gitlab.freedesktop.org/gstreamer/orc/-/merge_requests/234



I Upstreaming process

- GLib
 - GLib patches discarded by the community. Too intrusive
 - LLVM approach PR in review <https://github.com/llvm/llvm-project/pull/153168>
- GStreamer
 - Pending GLib support
- Cerbero
 - Pending installation support
- Orc
 - MR in progress

Real-life usage





| Projects

Fluendo was involved in the streaming and encoding/decoding process in the following projects:

- SPIRIT-STREAM: Scalable Telepresence with Real-time Enhanced Multimedia
- 6G-XR: Real-Time Holographic Communications



| Why gst.wasm?

- To try out new technologies like WebTransport, LCEVC, H.266
- Opportunity to test Fluendo H.266 Encoder in the web
- LCEVCdec (API) supports wasm
 - gst.wasm supports compiled LCEVCdec 3.x + SIMD.
 - LCEVCdec 4.x support remains pending.
- Opportunity to improve gst.wasm
- Opportunity to test gst.wasm in a real scenario



Public LCEVC-only gst.wasm demo:

<https://fluendo.github.io/gst.wasm/lcevcdec-example/lcevcdec-example.html>



SPRIT-STREAM: Scalable Telepresence with Real-time Enhanced Multimedia

- A user sends voice (5G)
- The end-user is presented to a pre-avatar.
- The server simulates mouth speaking movements in the avatar.
- The avatar is rendered in the browser (split rendering technology)
- The avatar is sent to the browser.



Funded by
the European Union

Project funded by



Schweizerische Eidgenossenschaft
Confédération suisse
Confederazione Svizzera
Confederaziun Svizra

Swiss Confederation

Federal Department of Economic Affairs,
Education and Research (EAF)
State Secretariat for Education,
Research and Innovation SERI



UK Research
and Innovation



SPIRIT-STREAM

- Fluendo selected to participate in the Open Call 1
- Key technology:
 - Decoder: Fluendo H.266 decoder (client/browser)
 - Encoder: Fluendo H.266 encoder (server/linux)
 - Protocol: QUIC/Webtransport
 - rtph266pay / rtph266depay

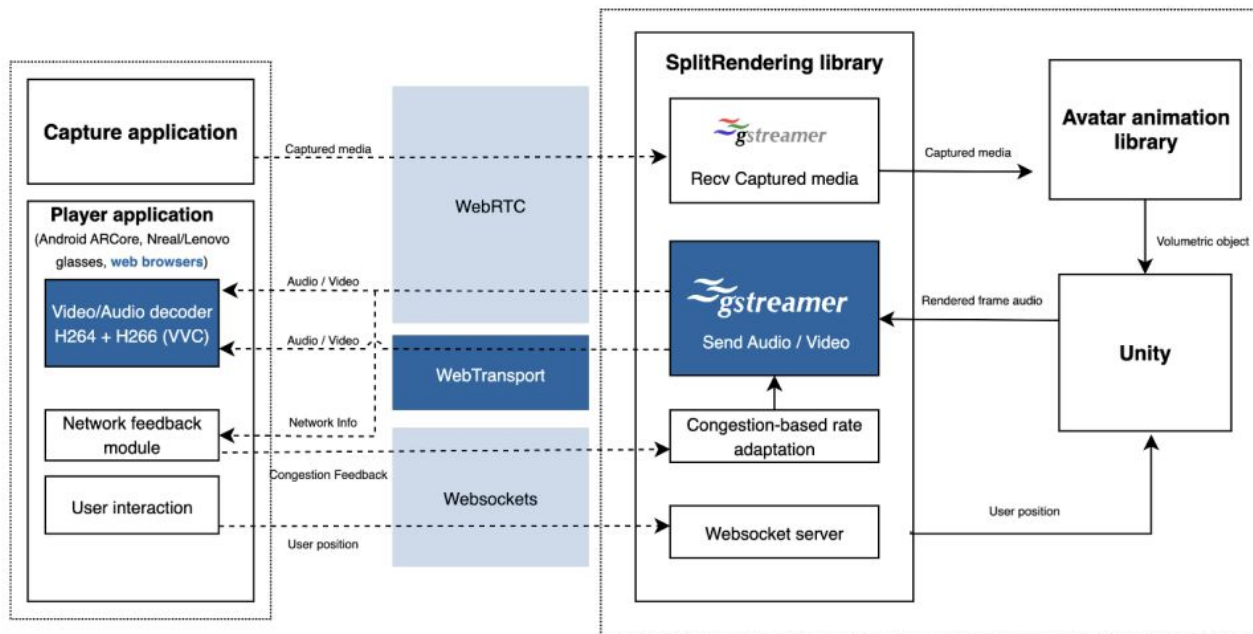


Funded by the European Union (SPIRIT, 101070672). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

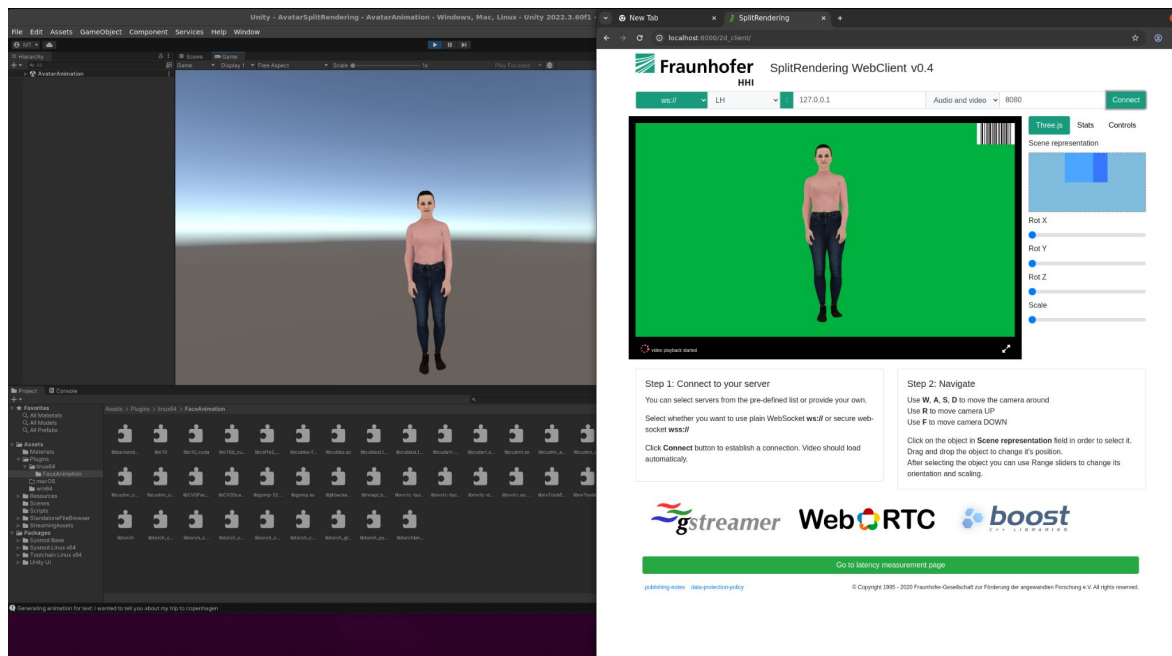
The SPIRIT project has received funding from the Swiss State Secretariat for Education, Research and Innovation (SERI).



SPIRIT-STREAM: Architecture



SPRIT-STREAM





I 6G-XR

- Funded project by the European Union
- 6G-XR develops an infrastructure for research in XR technologies
- Objectives
 - Build a multi-research infrastructure
 - Validate multi-access edge computing scenarios
 - Demonstrate and validate performance of innovative 6G applications
- Use cases
 - **Real-Time Holographic Communications (Fluendo involved)**
 - Collaborative 3D Digital Twin-like Environment
 - Energy Measurement Framework for Energy Sustainability



Co-funded by
the European Union





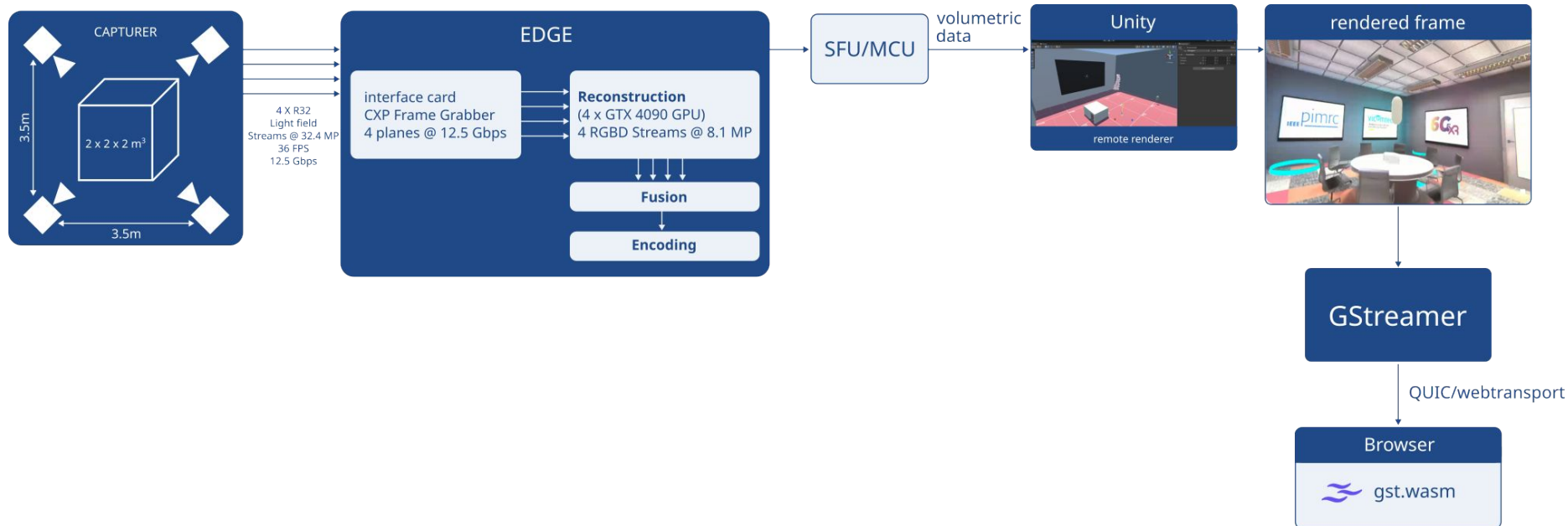
I 6G-XR: LCEVC-Powered holographic streaming

- Fluendo selected to participate in the Open Call 3 of 6G-XR
- Key technology:
 - Decoder: Icevcdec (client/browser)
 - Encoder: Fluendo Lynx (server/linux)
 - Protocol: WebTransport
- Infrastructure:
 - Server: 6G-XR South Node at i2CAT in Barcelona
 - Client: End-user laptop with a 6G modem at i2CAT offices



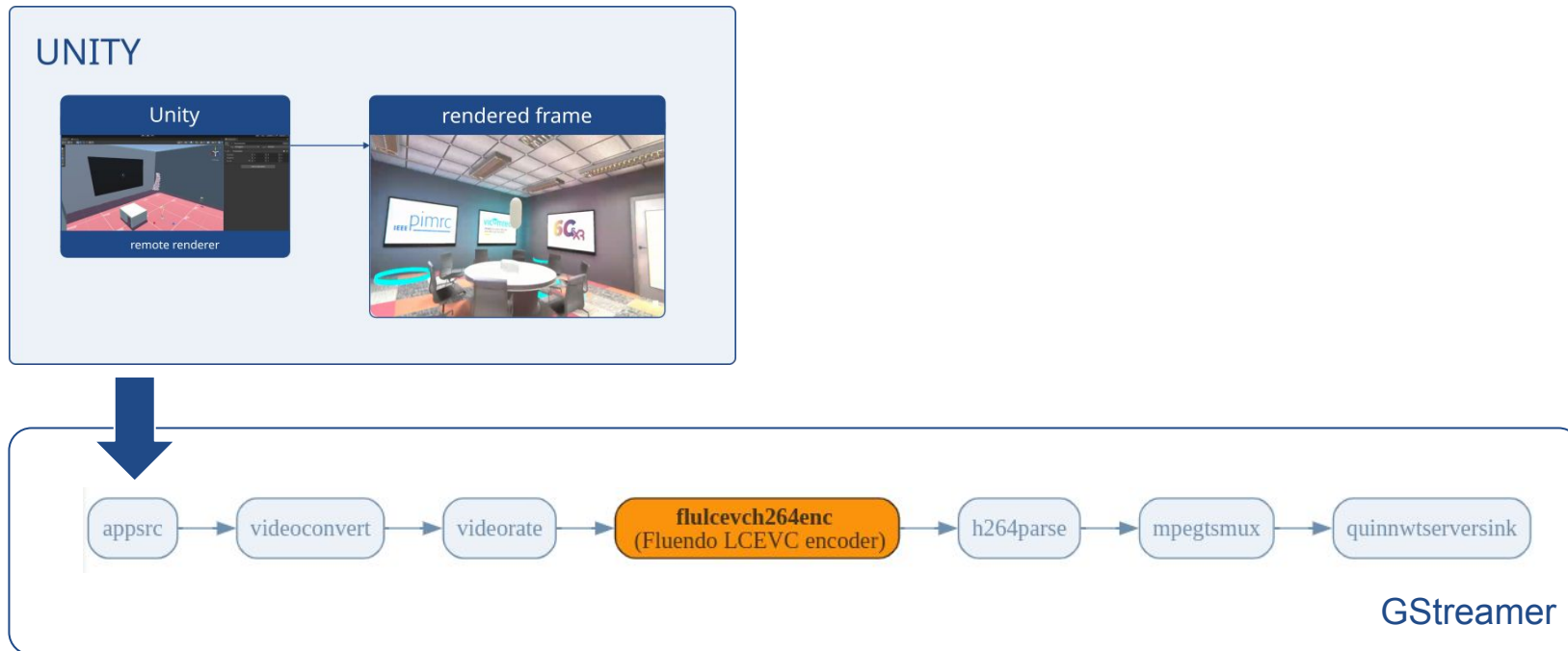
6G-XR project has received funding from the Smart Networks and Services Joint Undertaking (SNS JU) under the European Union's Horizon Europe research and innovation programme under Grant Agreement No 101096838. 6G-XR project has received funding from the Swiss State Secretariat for Education, Research and Innovation (SERI).

6G-XR: Architecture



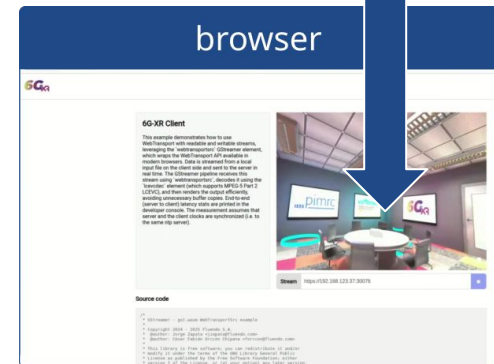
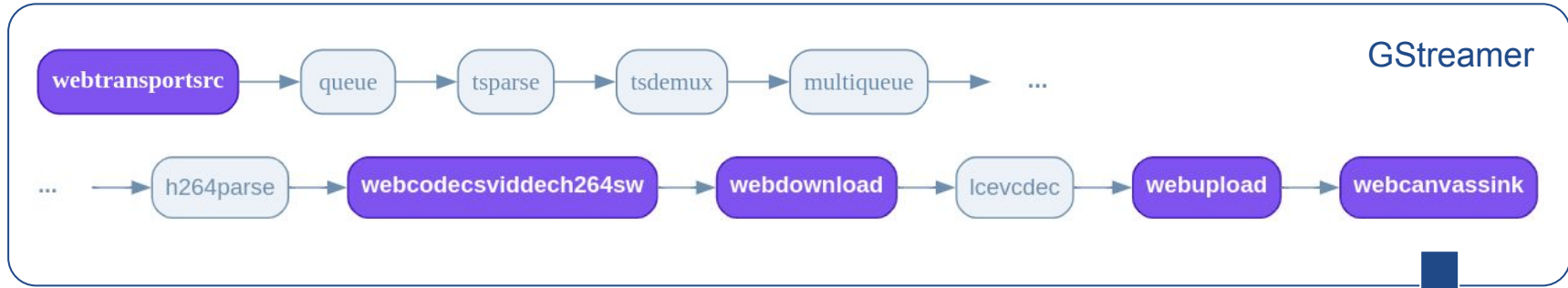


6G-XR: Server GStreamer pipeline (linux)





6G-XR: Client GStreamer pipeline (browser)





| 6G-XR: Browser configuration for WebTransport

- Import certificates
 - `chrome://settings -> Privacy and security -> Security -> Manage certificates -> Authorities -> Import -> Trust this certificate for identifying websites -> OK`
- Enable *`chrome://flags/#webtransport-developer-mode`*
- Enable *`chrome://flags/#enable-quic`*

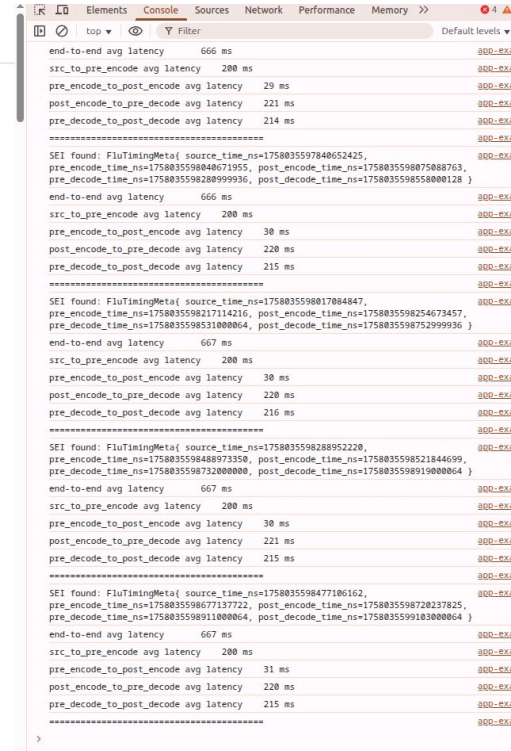
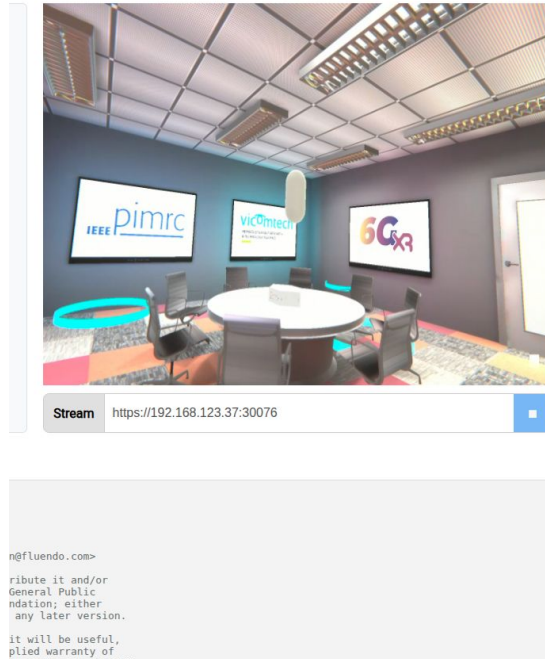


| 6G-XR: Achievements and limitations

- LCEVC on gst.wasm
 - GStreamer's LCEVC plug-in compiled for gst.wasm
- Audio
 - Added AAC decoder (**gstwebcodecsauddecaac[sw,hw]**) improvements
- Latency
 - Improved compiling LCEVC with WebAssembly SIMD
 - Improved implementing webupload as replacement of videoconvert
- End-to-end latency measurement
 - Inject SEI metadata with timestamps as frames leave the server and client (encoder, client source, decoder) to measure latency.
- GstNtpClock
 - Gather current time using GstNtpClock not possible because uses UDP sockets, not supported in Emscripten.



6G-XR: Client web page



Thank you!

Questions?

